DATA ENGINEERING (CO5173) – FINAL PROJECT
Search Techniques in Modern Database Systems: Elasticsearch & Cassandra

Supervisor: Assoc. Prof. Trần Minh Quang
Students: Đặng Kiến Tường (2670391) – Đặng Minh Quân (2670358)

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology (HCMUT)
Vietnam National University Ho Chi Minh City (VNU-HCM)

Semester 261 – Academic Year 2026–2027

Table of Contents

1. Introduction & Search Fundamentals
2. Elasticsearch: Distributed Full-Text Search
3. Apache Cassandra: Scalable Wide-Column Store
4. Comparative Analysis & Combined Architecture
5. Experimental Demo & Live Results
6. Conclusion



Table of Contents

1. Introduction & Search Fundamentals
2. Elasticsearch: Distributed Full-Text Search
3. Apache Cassandra: Scalable Wide-Column Store
4. Comparative Analysis & Combined Architecture
5. Experimental Demo & Live Results
6. Conclusion



Motivation: Why Search is Critical in Modern Systems

Modern Data Scale & User Expectations

- **E-Commerce Catalogs:** Millions of products; users expect sub-100ms responses, typo tolerance, faceted filters, and personalized ranking.
- **Observability & Logs:** Petabytes of semi-structured events requiring real-time ad-hoc text search and aggregations.
- **Social Media & Messaging:** Billions of messages indexed simultaneously by user/channel key and full-text content.

Why Traditional RDBMS Struggle

- `WHERE description LIKE '%laptop%'` forces a **full table scan** $O(N)$.
- B-trees only index exact prefixes (`'laptop%'`), not arbitrary tokens.
- No built-in **relevance scoring** (all matching rows returned with equal weight).
- Vertical scaling bottlenecks under heavy read/write concurrency.

Core Question: How do **Elasticsearch** (search engine) and **Cassandra** (wide-column store) solve retrieval at scale?



Search Fundamentals: Inverted Index & Ranking Formulas

Inverted Index Structure Zobel and Moffat, "Inverted Files for Text Search Engines"

- Maps each unique **term** (in a sorted Term Dictionary / FST) to a **posting list** of document IDs, term frequencies, and positions.

Term	DF	Posting List (Doc : TF)
`camera`	2	$[D_1 : 2,\ D_3 : 1]$
`laptop`	2	$[D_2 : 1,\ D_4 : 3]$
`waterproof`	1	$[D_1 : 1]$

- **FST (Finite State Transducer):** Compresses term dictionary in memory for fast prefix and fuzzy traversal.
- **Skip Lists / Roaring Bitmaps:** Accelerate boolean conjunctions ($\cap$) and disjunctions ($\cup$).

From TF-IDF to Okapi BM25 Robertson and Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond"

- **TF-IDF:** $\text{TF}(t, d) \times \text{IDF}(t)$ grows unboundedly with term frequency.
- **BM25** adds **term saturation** (k_1) and **document length normalization** (b):

$$\text{BM25}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\textbf{avgdl}}\right)}$$

$$\text{IDF}(t) = \ln\left(1 + \frac{N - n(t) + 0.5}{n(t) + 0.5}\right)$$

- $k_1 \approx 1.2$: controls TF saturation asymptote.
- $b \approx 0.75$: penalizes long documents ($|D| >$ avgdl).

Table of Contents

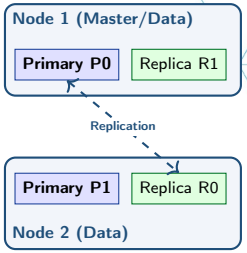
1. Introduction & Search Fundamentals
2. **Elasticsearch: Distributed Full-Text Search**
3. Apache Cassandra: Scalable Wide-Column Store
4. Comparative Analysis & Combined Architecture
5. Experimental Demo & Live Results
6. Conclusion



Elasticsearch Overview & Distributed Architecture

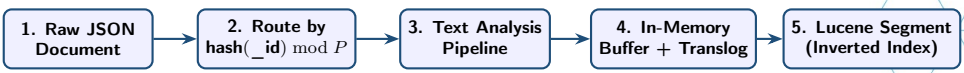
Core Architectural Concepts Gormley and Tong, *Elasticsearch: The Definitive Guide*; Elastic N.V., *Elasticsearch Reference 8.15*

- **Cluster & Nodes:** A collection of nodes coordinating storage and distributed search execution.
- **Index:** Logical namespace mapping to one or more physical shards.
- **Shard (Lucene Instance):** Self-contained Apache Lucene index consisting of immutable **segments**.
  - **Primary Shard:** Handles indexing writes first.
  - **Replica Shard:** Redundant copy for high availability and read throughput scaling.
- **Scatter–Gather Search:** Coordinating node broadcasts query to shards, merges top-K scores.



Each shard combines an Inverted Index (search) + Doc Values (columnar storage for aggregations).

Elasticsearch Indexing Pipeline



From JSON to Searchable Structures

1. **Document Ingestion:** JSON fields parsed according to index mapping (`text`, `keyword`, `float`).
2. **Analysis:** `text` fields pass through Character Filters → Tokenizer → Token Filters.
3. **Multi-Representation:**
   - **Inverted Index:** Token → Posting list.
   - **Doc Values:** Column-stride storage for sorting and aggregations.
   - **BKD Trees:** Multi-dimensional numeric/range indexing.

Near Real-Time (NRT) Refresh & Durability

- **In-Memory Buffer + Translog:** New documents are appended to the write-ahead `translog` for crash recovery and buffered in RAM.
- **Refresh (every 1s):** Buffer is written to a new immutable **Lucene segment** in OS page cache → document becomes searchable.
- **Flush & Segment Merge:** Periodically commits segments to disk (`fsync`), clears translog, and merges smaller segments in the background.

Elasticsearch Analyzers Deep Dive

Custom Analyzers (demo/es/index_mapping.json):

```
"analysis": {
  "analyzer": {
    "english_custom": {
      "type": "custom",
      "tokenizer": "standard",
      "filter": ["lowercase", "english_stemmer",
           "english_stop"]
    },
    "autocomplete": {
      "type": "custom",
      "tokenizer": "standard",
      "filter": ["lowercase", "edge_ngram_2_15"]
    }
  },
  "filter": {
    "english_stemmer": { "type": "stemmer", "language":
         "english" },
    "english_stop": { "type": "stop", "stopwords":
         "_english_" },
    "edge_ngram_2_15": { "type": "edge_ngram",
         "min_gram": 2, "max_gram": 15 }
  }
}
```

Demo 9: `_analyze` API Transformation

- **Input:** `"The professional cameras are featuring waterproof housings"`
- **1. Standard Tokenizer:** Splits on word boundaries.
- **2. Lowercase + Stop Filter:** Removes `"The"`.
- **3. English Stemmer:** Reduces words to root stems:

```
["profession", "camera", "ar",
 "featur", "waterproof", "hous"]
```

Autocomplete (edge_ngram_2_15):

- `"Samsung"` → `["sa", "sam", "sams", "samsu", "samsun", "samsung"]` at write time.
- `search_analyzer: "standard"` keeps queries fast.

Elasticsearch Search Types – Part 1: Text, Fuzzy & Autocomplete

1. Full-Text `match` Query (Demo 1)

```
{ "size": 3,
  "query": {
    "match": {
      "description": "lightweight laptop for travel"
    }
  },
  "_source": ["name", "category", "price",
       "rating"] }
```

- Analyzes query into stemmed tokens; ranks 2,042 matching docs via BM25 (`max_score: 5.77`).

3. Edge N-Gram Autocomplete (Demo 3)

```
{ "size": 5,
  "query": {
    "match": { "name.autocomplete": "sams" }
  },
  "_source": ["name", "brand", "category"] }
```

2. Fuzzy Multi-Match Query (Demo 2)

```
{ "size": 3,
  "query": {
    "multi_match": {
      "query": "lapptop wireles",
      "fields": ["name^2", "description"],
      "fuzziness": "AUTO"
    }
  },
  "_source": ["name", "category", "price"] }
```

How Fuzzy & Field Boosting Work

- `fuzziness: "AUTO"` allows Levenshtein edit distance 1 (3–5 chars) or 2 (> 5 chars) via Levenshtein automata over the FST.
- `"lapptop"` $\rightarrow$ `"laptop"` ($d = 1$); `"wireles"` $\rightarrow$ `"wireless"` ($d = 1$).
- `name^2` doubles relevance weight of title matches.

Elasticsearch Search Types – Part 2: Bool, Aggregations & Function Score

4. Compound `bool` Query (Demo 4)

```
"query": { "bool": {
  "must": [
    { "match": { "description": "noise cancelling"
        } }
  ],
  "filter": [
    { "term":  { "category": "headphones" } },
    { "range": { "price":  { "lte": 100 } } },
    { "range": { "rating": { "gte": 4.0 } } }
  ]
}}
```

- **Query Context (`must`):** Computes BM25 score.
- **Filter Context (`filter`):** Binary yes/no (0 score overhead), cached in bitsets for fast reuse.

5. Faceted Aggregations (Demo 5)

```
"size": 0,
"aggs": { "by_category": {
  "terms": { "field": "category" },
  "aggs": {
    "avg_price":  { "avg": { "field": "price" } },
    "max_rating": { "max": { "field": "rating" } }
  }
}}
```

6. Custom Ranking (`function_score`, Demo 6)

```
"query": { "function_score": {
  "query": { "match": { "description": "camera" } },
  "functions": [
    { "field_value_factor": { "field": "rating",
        "factor": 1.2, "modifier": "sqrt" } },
    { "field_value_factor": { "field":
        "review_count", "factor": 0.001,
        "modifier": "log1p" } }
  ],
  "boost_mode": "multiply"
}}
```

Elasticsearch Relevance & Scoring: Explain API & Highlighting

Demo 8: `explain: true` **for** `"espresso grinder"`

```
"_id": "79", "_score": 2.6103396,
"_source": { "name": "Nespresso MokaPro 440" },
"_explanation": {
  "value": 2.6103396,
  "description": "weight(description:grinder in 78):",
  "details": [{
    "value": 2.6103396,
    "description": "score(freq=1.0), boost * idf * tf:",
    "details": [
      { "value": 2.2, "description": "boost" },
      { "value": 2.5000172,
        "description": "idf, log(1+(N-n+0.5)/(n+0.5))",
        "details": [{"value": 410, "description": "n"},
                    {"value": 5000, "description": "N"}]
                        },
      { "value": 0.47460395,
        "description": "tf,
             freq/(freq+k1*(1-b+b*dl/avgdl))",
        "details": [{"value": 1.0, "description":
             "freq"},
                    {"value": 14.0, "description": "dl"},
                    {"value": 15.613, "description":
                         "avgdl"}] }
    ]
  }]
}
```

Exact BM25 Score Verification:

$$\text{IDF} = \ln\left(1 + \frac{5000 - 410 + 0.5}{410 + 0.5}\right) = 2.5000$$

$$\text{TF}_{\text{norm}} = \frac{1.0}{1.0 + 1.2\left(0.25 + 0.75\frac{14.0}{15.613}\right)} = 0.4746$$

$$\text{Score} = 2.2 \times 2.5000 \times 0.4746 = \mathbf{2.6103}$$

Demo 7: Search Highlighting

```
"highlight": { "fields": { "description": {} } }
// Output fragment:
"The Fujifilm AlphaShot 543 is a professional
 <em>camera</em> (<em>action</em>
      <em>camera</em>)
 featuring optical zoom 30x and
      <em>waterproof</em>"
```

Table of Contents

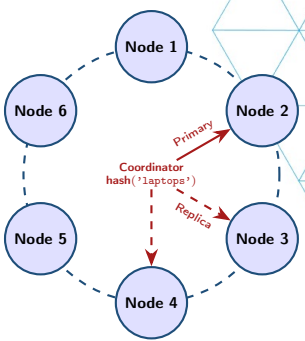
1. Introduction & Search Fundamentals
2. Elasticsearch: Distributed Full-Text Search
3. **Apache Cassandra: Scalable Wide-Column Store**
4. Comparative Analysis & Combined Architecture
5. Experimental Demo & Live Results
6. Conclusion



Apache Cassandra Overview: Masterless Ring Architecture

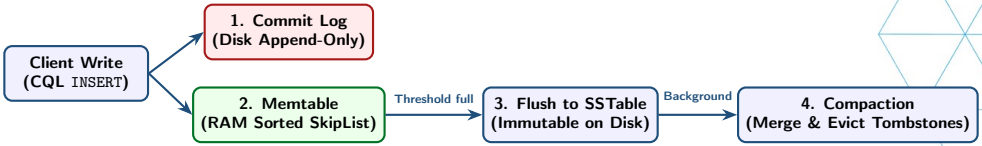
Decentralized Peer-to-Peer Design Lakshman and Malik, "Cassandra: A Decentralized Structured Storage System"; Karger et al., "Consistent Hashing and Random Trees"

- **Masterless Ring Topology:** Every node plays an identical role; no single point of failure (SPOF).
- **Consistent Hashing:** Partition key is hashed via **Murmur3Partitioner** (range $[-2^{63}, 2^{63}-1]$) to determine token ownership on the ring.
- **Virtual Nodes (vnodes):** Each physical node owns multiple token ranges (e.g., 16–256 vnodes) for balanced data distribution and seamless cluster resizing.
- **Coordinator Node:** Any node can receive a client request and route it directly in $O(1)$ hops to the replica nodes holding that token.
- **Tunable Consistency:** Configure `ONE`, `QUORUM`, or `ALL` per query ($R + W >$ RF guarantees strong consistency).



Replication Factor (RF=3): token is replicated clockwise around the consistent hash ring.

Cassandra Write Path & LSM-Tree Storage Engine



Why Writes Are Ultra-Fast ($O(1)$ Disk I/O)

- **No Read-Before-Write:** Unlike B-Trees, Cassandra never seeks or overwrites existing pages in place O'Neil et al., "The Log-Structured Merge-Tree (LSM-Tree)".
- **Dual Write:** Sequential append to the **Commit Log** (durability) + insertion into the in-memory **Memtable** sorted by clustering key.

SSTables, Tombstones & Compaction

- **SSTable (Sorted String Table):** Flushed sequentially to disk; completely immutable.
- **Deletes = Tombstones:** Marker written with timestamp; reclaimed after `gc_grace_seconds`.
- **Compaction:** Periodically merges multiple SSTables into one, keeping only the latest cell timestamp.

Cassandra Read Path: Multi-Layer Filtering & Merge



How Cassandra Avoids Reading Every SSTable

1. **Bloom Filter (RAM):** Probabilistic bit array per SSTable. Guarantees *no false negatives* — immediately skips SSTables that definitely do not contain the partition key.
2. **Key Cache & Partition Summary:** Jumps directly to the byte offset in the Partition Index without extra disk seeks.
3. **Clustering Slice Optimization:** Min/max clustering column metadata skips SSTables outside the requested range (e.g., `price BETWEEN 500 AND 800`).

Demo 8: Live `TRACING ON` Output

- `Executing single-partition query on products_by_category [ReadStage-1]` (25.7 ms)
- `Skipped 0/0 non-slice-intersecting sstables` (27.0 ms)
- `Merged data from memtables and 0 sstables` (29.4 ms)
- `Read 3 live rows and 0 tombstone cells` (29.8 ms)

Cassandra Query-First Data Modeling

Schema Definition (`demo/cassandra/schema.cql`):

```
CREATE KEYSPACE IF NOT EXISTS shop
  WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
  };

CREATE TABLE IF NOT EXISTS shop.products_by_category (
  category      text,
  price         decimal,
  product_id    int,
  name          text,
  brand         text,
  description   text,
  rating        float,
  stock         int,
  review_count  int,
  PRIMARY KEY ((category), price, product_id)
) WITH CLUSTERING ORDER BY (price ASC, product_id ASC);
```

Anatomy of the Compound Primary Key

`PRIMARY KEY ((category), price, product_id)`

- **Partition Key** (`category`): Determines *which node/partition* stores the data via Murmur3(`category`). All laptops live together in one physical partition.
- **Clustering Columns** (`price, product_id`): Determines the *on-disk sort order* within that partition.
- `product_id` acts as tie-breaker so two products with the same price don't overwrite each other.

Result: Category browse + price range filter is a **single sequential disk read** with zero secondary index overhead!



Cassandra 5.0 Storage-Attached Indexes (SAI)

SAI Definitions in `schema.cql`:

```
CREATE CUSTOM INDEX IF NOT EXISTS idx_brand
  ON shop.products_by_category (brand)
  USING 'StorageAttachedIndex';

CREATE CUSTOM INDEX IF NOT EXISTS idx_rating
  ON shop.products_by_category (rating)
  USING 'StorageAttachedIndex';
```

How SAI Works Under the Hood Apache Software Foundation, *Storage-Attached Indexing (SAI) in Cassandra 5.0*

- **SSTable-Attached Lifecycle:** Index files are written alongside SSTable flushes and merged during compaction.
- **Strings (`brand`):** Indexed via memory-efficient **Byte-Ordered Tries**.
- **Numerics (`rating`):** Indexed via balanced **BKD Trees** for range lookups.

Legacy 2i vs. Cassandra 5.0 SAI

Feature	Legacy 2i	SAI (Cassandra 5.0)
Storage Engine	Hidden CQL table	Attached to SSTable
Write Overhead	High (extra write path)	Low (shared flush)
Numeric Ranges	Poor support	Fast (BKD trees)
Index Footprint	Large disk usage	60–80% smaller
Multi-Index `AND`	Single index + filter	Intersects posting lists

Important Caveat

Without a partition key (`WHERE brand='Sony'`), SAI still performs a **scatter-gather** across ring nodes, though local SSTable lookup is much faster than full scan.



Cassandra Limitations for Full-Text Search

1. Refusal on Unindexed Columns (Demo 5)

```
SELECT name FROM shop.products_by_category
WHERE stock > 100 LIMIT 5;
-- InvalidRequest: Cannot execute this query as it
-- might involve data filtering... use ALLOW FILTERING
```

2. The `ALLOW FILTERING` Anti-Pattern (Demo 6)

```
SELECT name, stock FROM shop.products_by_category
WHERE stock > 100 LIMIT 5 ALLOW FILTERING;
```

- Works on 5,000 demo rows, but in production forces coordinator to scan **every SSTable across every node**.

3. No `LIKE '%...%'` Substring Search (Demo 7)

```
SELECT name FROM shop.products_by_category
WHERE description LIKE '%laptop%' LIMIT 3;
-- InvalidRequest: LIKE restriction is only supported
-- on properly indexed columns.
```

What Cassandra Cannot Do Natively

- **No Relevance Ranking (BM25):** Rows are returned strictly in token or clustering order — no concept of "best match".
- **No Fuzzy / Typo Tolerance:** Cannot match `"lapptop"` to `"laptop"`.
- **No Stemming or Synonyms:** `"cameras"` will not match `"camera"` without custom analyzer pipelines.
- **No Faceted Aggregations:** Computing `GROUP BY category` with `AVG(price)` across partitions is an anti-pattern.

Key Insight: Cassandra's refusal is a *feature*, protecting cluster SLA from unbounded latency.



Table of Contents

1. Introduction & Search Fundamentals
2. Elasticsearch: Distributed Full-Text Search
3. Apache Cassandra: Scalable Wide-Column Store
4. **Comparative Analysis & Combined Architecture**
5. Experimental Demo & Live Results
6. Conclusion

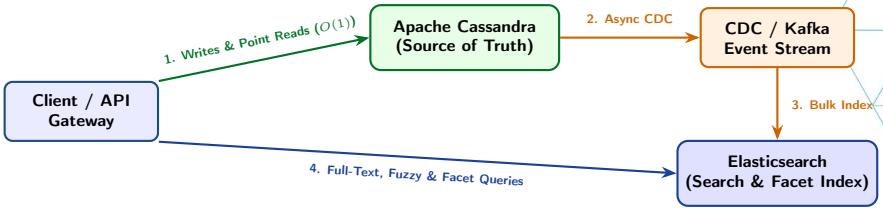


Elasticsearch vs. Apache Cassandra: Side-by-Side Comparison

Dimension	**Elasticsearch 8.15** Elastic N.V., *Elasticsearch Reference 8.15*	**Apache Cassandra 5.0** Apache Software Foundation, *Apache Cassandra 5.0 Documentation*
Primary Paradigm	Distributed Search & Analytics Engine	Wide-Column Distributed NoSQL Store
Core Data Structure	Lucene Inverted Index + FST + Doc Values + BKD Trees	Partitioned LSM-Tree (Memtable + Immutable SSTables) + SAI
Query Capabilities	Full-text, BM25 ranking, fuzzy (Levenshtein), edge n-gram autocomplete, nested aggregations	$O(1)$ partition key lookup, clustering range scans, exact/range SAI filtering
Data Modeling	Schema-flexible JSON; index all searchable fields (*Document-First*)	Denormalized tables per access pattern (*Query-First*)
Write Performance	Medium–High (CPU tokenization/indexing, NRT 1s refresh delay)	Ultra-High (append-only commit log + RAM memtable, immediate visibility)
Scalability & Topology	Dedicated Master/Data nodes; shard rebalancing required when scaling	Masterless consistent-hashing ring with vnodes; linear horizontal write scaling
Consistency Model	Eventual read visibility (1s refresh); primary → replica sync writes	Tunable per operation (`ONE`, `LOCAL_QUORUM`, `ALL`)
Ideal Use Case	Product discovery, log analytics, ad-hoc user queries, faceted navigation	High-velocity writes, source-of-truth order/catalog storage, time-series



Combined Production Architecture: Best of Both Worlds



Synchronization Patterns

- **Change Data Capture (CDC) + Kafka:** Cassandra commits writes first; CDC connector streams mutations to Kafka, which bulk-indexes into Elasticsearch.
- **Application Dual-Write:** Simpler for small systems, but requires idempotent retries to prevent drift.

Industry Examples (Netflix, Discord, eBay)

- **Discord / E-Commerce Pattern:**
  1. User searches `"noise cancelling headphones"` → ES returns ranked `product_ids` & facets.
  2. Checkout/Inventory service queries **Cassandra** by key for authoritative stock and price.

Table of Contents

1. Introduction & Search Fundamentals
2. Elasticsearch: Distributed Full-Text Search
3. Apache Cassandra: Scalable Wide-Column Store
4. Comparative Analysis & Combined Architecture
5. **Experimental Demo & Live Results**
6. Conclusion



Experimental Demo Setup: E-Commerce Product Catalog

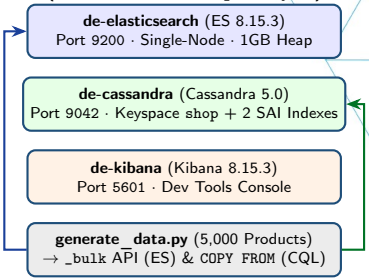
Synthetic Dataset (`generate_data.py`, seed $= 42$)

- **Volume:** $N = 5,000$ synthetic products loaded identically into `products_bulk.ndjson` (ES) and `products.csv` (Cassandra).
- **9 Attributes:** `product_id`, `name`, `category`, `brand`, `description`, `price`, `rating`, `stock`, `review_count`.

Category	Price ($)	Sample Brands
laptops	450–3,500	Dell, Asus, Lenovo, Apple, HP, MSI
smartphones	200–1,800	Samsung, Apple, Xiaomi, Sony, Google
headphones	25–550	Sony, Bose, JBL, Anker, Sennheiser
cameras	150–4,200	Canon, Nikon, Sony, Fujifilm, GoPro
coffee machines	40–1,200	DeLonghi, Breville, Philips, Nespresso
running shoes	35–280	Nike, Adidas, Asics, New Balance, Hoka

Docker Compose Stack

(`demo/docker-compose.yml`)



Live Demo: Elasticsearch Full-Text (BM25) & Fuzzy Search

Demo 1: Full-Text Search (BM25 Ranking)

```
POST /products/_search
{ "size": 3,
  "query": { "match": {
    "description": "lightweight laptop for travel"
  }},
  "_source": ["name", "category", "price", "rating"]
       }
```

Real Execution Output (2,042 hits, 50 ms):

```
"hits": [
  { "_id": "958",  "_score": 5.7693315,
    "_source": { "name": "HP WorkStation 438",
      "category": "laptops", "price": 2594.98,
           "rating": 5.0 } },
  { "_id": "1067", "_score": 5.7693315,
    "_source": { "name": "Apple WorkStation 842",
      "category": "laptops", "price": 1672.26,
           "rating": 4.4 } },
  { "_id": "1587", "_score": 5.7693315,
    "_source": { "name": "Lenovo GameMaster 592",
      "category": "laptops", "price": 2527.12,
           "rating": 4.0 } }
]
```

Demo 2: Fuzzy Search with Typos

```
POST /products/_search
{ "size": 3,
  "query": { "multi_match": {
    "query": "lapptop wireles",
    "fields": ["name^2", "description"],
    "fuzziness": "AUTO"
  }},
  "_source": ["name", "category", "price"] }
```

Real Execution Output (843 hits, 48 ms):

```
"hits": [
  { "_id": "65",  "_score": 1.507406,
    "_source": { "name": "HP AirLite 720",
      "category": "laptops", "price": 3073.51 } },
  { "_id": "168", "_score": 1.507406,
    "_source": { "name": "HP UltraBook 295",
      "category": "laptops", "price": 1709.53 } },
  { "_id": "262", "_score": 1.507406,
    "_source": { "name": "Asus ProBook 923",
      "category": "laptops", "price": 2809.26 } }
]
```



Live Demo: Elasticsearch Bool Filters & Faceted Aggregations

Demo 4: Bool Query + Structured Filters

```
{ "size": 3,
  "query": { "bool": {
    "must": [{ "match": {"description": "noise
         cancelling"} }],
    "filter": [
      { "term":  { "category": "headphones" } },
      { "range": { "price":  { "lte": 100 } } },
      { "range": { "rating": { "gte": 4.0 } } }
    ]
  }},
  "_source": ["name", "price", "rating"] }
```

Real Execution Output (14 hits, 28 ms):

```
"hits": [
  { "_id": "1058", "_score": 6.52806,
    "_source": { "name": "Anker QuietComfort 535",
                 "price": 43.28, "rating": 4.6 } },
  { "_id": "1717", "_score": 6.52806,
    "_source": { "name": "Anker SoundCore 251",
                 "price": 80.98, "rating": 4.1 } },
  { "_id": "2604", "_score": 6.52806,
    "_source": { "name": "Sennheiser StudioPro 709",
                 "price": 79.21, "rating": 4.2 } }
]
```

Demo 5: Faceted Aggregations (doc_values)

```
{ "size": 0,
  "aggs": { "by_category": {
    "terms": { "field": "category" },
    "aggs": {
      "avg_price":  { "avg": { "field": "price" } },
      "max_rating": { "max": { "field": "rating" } }
    }
  }} }
```

Real Execution Output (All 5,000 docs):

Category Bucket	Doc Count	Avg Price ($)	Max Rating
running shoes	857	159.89	5.0
cameras	855	2,152.66	5.0
laptops	843	1,927.43	5.0
coffee machines	838	612.65	5.0
smartphones	832	992.26	5.0
headphones	775	287.14	5.0



Live Demo: Cassandra Partition Key, Clustering Range & SAI

Demo 1: Partition-Key Lookup ($O(1)$ Routing)

```
CQL> SELECT name, price, rating
     FROM shop.products_by_category
     WHERE category='laptops' LIMIT 5;

 name                | price  | rating
---------------------+--------+--------
      HP AirLite 555 | 454.77 |    3.9
    Dell AirLite 469 | 455.93 |    4.5
   Apple AirLite 876 |  462.8 |    3.8
 MSI GameMaster 540  | 466.59 |    4.1
    Dell ProBook 703 | 471.81 |    3.8
```

Demo 2: Partition + Clustering Range

```
CQL> SELECT name, price FROM
     shop.products_by_category
     WHERE category='laptops'
       AND price>=500 AND price<=800 LIMIT 5;

 name                  | price
-----------------------+--------
      Dell AirLite 207 | 502.67
    Dell UltraBook 557 | 509.93
      HP UltraBook 523 | 510.02
```

Demo 3: SAI Equality Index on `brand`

```
CQL> SELECT name, category, price
     FROM shop.products_by_category
     WHERE brand='Sony' LIMIT 5;

 name                | category     | price
---------------------+--------------+--------
     Sony Galaxy 824 | smartphones  |  202.1
  Sony RedMagic 574  | smartphones  |  210.2
    Sony Galaxy 930  | smartphones  | 248.77
    Sony Xperia 893  | smartphones  |  252.4
 Sony Pixel Pro 948  | smartphones  | 270.12
```

Demo 4: Partition Key + SAI Numeric Range

```
CQL> SELECT name, rating, price
     FROM shop.products_by_category
     WHERE category='laptops' AND rating>=4.5 LIMIT
          5;

 name                   | rating | price
------------------------+--------+--------
       Dell AirLite 469 |    4.5 | 455.93
        MSI ProBook 984 |    4.9 | 477.91
 Apple WorkStation 852  |    4.9 | 481.92
```

Live Demo: Cassandra Anti-Patterns & Search Failures

1. Filtering Without Index → Refused! (Demo 5)

```
CQL> SELECT name FROM shop.products_by_category
     WHERE stock > 100 LIMIT 5;

InvalidRequest: Error from server: code=2200 [Invalid
     query]
message="Cannot execute this query as it might involve
     data
filtering and thus may have unpredictable performance.
     If you
want to execute this query despite the performance
unpredictability, use ALLOW FILTERING"
```

2. Full-Text `LIKE` Pattern → Refused! (Demo 7)

```
CQL> SELECT name FROM shop.products_by_category
     WHERE description LIKE '%laptop%' LIMIT 3;

InvalidRequest: Error from server: code=2200 [Invalid
     query]
message="LIKE restriction is only supported on properly
indexed columns. description LIKE '%laptop%' is not
     valid."
```

3. Forcing `ALLOW FILTERING` (Demo 6)

```
CQL> SELECT name, stock
     FROM shop.products_by_category
     WHERE stock > 100 LIMIT 5
     ALLOW FILTERING;

 name                          | stock
-------------------------------+-------
 Breville BrewPerfect 454 |    488
 DeLonghi BrewPerfect 161 |    175
  Philips BrewPerfect 968 |    332
    Nespresso MokaPro 354 |    456
     Breville MokaPro 381 |    253
```

Why This Matters in Production

- `ALLOW FILTERING` scans all partitions until `LIMIT` 5 is satisfied (or scans 100% of the cluster if < 5 rows match).
- **Takeaway:** Never use Cassandra for ad-hoc text search; pair it with Elasticsearch!

Table of Contents

1. Introduction & Search Fundamentals
2. Elasticsearch: Distributed Full-Text Search
3. Apache Cassandra: Scalable Wide-Column Store
4. Comparative Analysis & Combined Architecture
5. Experimental Demo & Live Results
6. **Conclusion**



Conclusion & Key Takeaways

1. Elasticsearch = Discovery & Relevance

- Answers **unstructured, user-typed queries** via Lucene inverted indexes, custom analyzers (stemming, stopwords, edge n-grams), and FSTs.
- Provides **Okapi BM25 ranking**, typo-tolerant fuzzy search, highlighting, and real-time faceted aggregations via columnar `doc_values`.

2. Cassandra = Predictable Scale

- Answers **known, structured access patterns** in $O(1)$ network hops via consistent hashing and sorted LSM-Tree partitions.
- Cassandra 5.0 **SAI** enables efficient secondary attribute filtering, but purposefully omits full-text relevance ranking.

3. Architectural Synthesis

- **Right Tool for the Job:** Neither database replaces the other; their internal data structures (*Inverted Index vs. Partitioned LSM-Tree*) are optimized for fundamentally different workloads.
- **Production Standard:** Combine **Cassandra as the durable source of truth** with **Elasticsearch as the search index**, synchronized asynchronously via Change Data Capture (CDC) and Kafka.

Reproducible Artifact: Full Docker stack, 5,000-product generator, ES mappings, CQL schema, and 17 live demo queries included in repository.



References I

- Apache Software Foundation. *Apache Cassandra 5.0 Documentation*. 2024. URL: https://cassandra.apache.org/doc/latest/.
- — .*Storage-Attached Indexing (SAI) in Cassandra 5.0*. 2024. URL: https://cassandra.apache.org/doc/latest/cassandra/indexing/sai/sai-overview.html.
- Elastic N.V. *Elasticsearch Reference 8.15*. 2024. URL: https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html.
- Gormley, Clinton and Zachary Tong. *Elasticsearch: The Definitive Guide*. O'Reilly Media, 2015.
- Karger, David et al. "Consistent Hashing and Random Trees". In: (1997).
- Lakshman, Avinash and Prashant Malik. "Cassandra: A Decentralized Structured Storage System". In: *ACM SIGOPS Operating Systems Review*. Vol. 44. 2. 2010, pp. 35–40.
- O'Neil, Patrick et al. "The Log-Structured Merge-Tree (LSM-Tree)". In: vol. 33. 4. 1996, pp. 351–385.

References II

- Robertson, Stephen and Hugo Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.
- Zobel, Justin and Alistair Moffat. "Inverted Files for Text Search Engines". In: vol. 38. 2. 2006.



- THANK YOU -

Questions & Answers (Q&A)

Course: Data Engineering (CO5173) – Semester 261
Supervisor: Assoc. Prof. Trần Minh Quang
Students: Đặng Kiến Tường (2670391) – Đặng Minh Quân (2670358)
Live Demo: Elasticsearch (:9200) · Kibana (:5601) · Cassandra 5.0 (:9042)